

From Zero to Golden: A Self-Validating Multi-Agent Framework to Solve the Text-to-SQL Cold Start

(November 2025)

Nguyen Thanh Canh¹, Ho Thi Nhu Tam¹ and Tran Thi Anh Thi^{1*}

Abstract—The adoption of Text-to-SQL in enterprises faces a major barrier: data cost. The most powerful AI models require extensive training on expensive “golden data,” which must be custom-built for each application’s database schema. In the absence of data, off-the-shelf solutions such as static system prompts, while incurring operational costs, yield unstable results with low accuracy and fail to accumulate value over time. To resolve the paradox that “without data, the system cannot operate, and without operation, there is no data,” we introduce Trustay-AI — an intelligent AI architecture that requires no prior training, utilizing four coordinated agents to accurately simulate expert reasoning: an Orchestrator Agent for intent understanding, a SQL Generation Agent with self-correction capabilities, a Response Generator Agent to format results into user-friendly outputs, and a Validator Agent to ensure the final result is logically correct. The core value of Trustay-AI lies in its “knowledge harvesting” capability — each SQL query validated as correct by the Validator Agent is automatically stored in a canonical SQL knowledge repository, not only enabling faster responses for subsequent queries but, more importantly, serving as valuable data assets that accumulate over time. Our experiments demonstrate that this architecture achieves an impressive accuracy of approximately 80.0%, significantly surpassing the 45.8% baseline achieved by conventional static system prompt approaches. This accumulated canonical SQL repository constitutes the “golden data” that enterprises can leverage to fine-tune specialized models in the future, thoroughly addressing the challenges of cost and dependency on initial training data.

Index Terms— Text-to-SQL, Multi-Agent System, Retrieval-Augmented Generation, Self-Validation, Data Bootstrapping, Cold Start.

Tóm tắt nội dung— Việc áp dụng Text-to-SQL trong doanh nghiệp đang đối mặt với một rào cản lớn: chi phí dữ liệu. Các mô hình AI mạnh mẽ nhất đòi hỏi phải được huấn luyện rất kỹ trên một kho “dữ liệu vàng” (golden data) đắt đỏ, vốn phải được xây dựng riêng cho database schema của từng ứng dụng. Khi thiếu dữ liệu, các giải pháp sẵn có như system prompt tĩnh tuy tiêu tốn chi phí vận hành nhưng kết quả lại thiếu ổn định, độ chính xác thấp và không mang lại giá trị tích lũy. Để giải quyết nghịch lý “không có dữ liệu thì không vận hành được, mà không vận hành được thì không có dữ liệu”, chúng tôi giới thiệu Trustay-AI — một kiến trúc AI thông minh không yêu

cầu huấn luyện trước, sử dụng bốn agent phối hợp với nhau để mô phỏng chính xác cách một chuyên gia suy luận: một Orchestrator Agent hiểu ý định, một SQL Generation Agent có khả năng tự sửa lỗi, một Response Generator Agent để định dạng kết quả thành các đầu ra thân thiện với người dùng, và một Validator Agent để đảm bảo kết quả cuối cùng đúng về mặt logic. Giá trị cốt lõi của Trustay-AI nằm ở khả năng “thu hoạch tri thức” — mỗi câu SQL được Validator Agent kiểm định là đúng sẽ được tự động cất vào một kho tri thức SQL chuẩn (canonical SQL), không chỉ giúp trả lời nhanh hơn cho các lần hỏi sau mà quan trọng hơn, nó chính là tài sản dữ liệu quý giá được tích lũy dần theo thời gian. Thực nghiệm của chúng tôi chứng minh rằng kiến trúc này đạt độ chính xác ấn tượng khoảng 80.0%, vượt xa mức 45.8% của phương pháp baseline (System Prompt Tĩnh) thông thường. Bộ canonical SQL được tích lũy này chính là “dữ liệu vàng” mà doanh nghiệp có thể dùng để fine-tune các mô hình chuyên sâu trong tương lai, giải quyết triệt để bài toán chi phí và sự phụ thuộc vào dữ liệu huấn luyện ban đầu.

Từ khóa— Text-to-SQL, Hệ thống đa tác nhân, Tạo sinh Tăng cường Truy xuất (RAG), Tự xác thực, Tạo Dữ liệu Vàng, Khởi động lạnh.

I. GIỚI THIỆU

Trong bối cảnh các hệ thống truy cập dữ liệu bằng ngôn ngữ tự nhiên ngày càng phổ biến, bài toán Text-to-SQL gặp phải những thách thức nghiêm trọng về độ chính xác ngữ cảnh, lỗi logic và thiếu cơ chế kiểm định, dẫn đến các truy vấn sai được thực thi trực tiếp trên cơ sở dữ liệu production.

Trong thực tế, việc dựng một mô hình Text-to-SQL gặp phải nhiều thách thức nghiêm trọng. Đầu tiên, các nhà phát triển thường phải huấn luyện một mô hình AI từ đầu, một quá trình tốn kém về thời gian và tài nguyên tính toán. Hơn nữa, khi dữ liệu thay đổi hoặc schema cơ sở dữ liệu được cập nhật, mô hình cần được huấn luyện lại hoàn toàn để thích ứng với bối cảnh mới, dẫn đến chi phí vận hành cao và khó khăn trong việc duy trì tính cập nhật của hệ thống. Sau khi phân tích kỹ càng, chúng tôi nhận thấy nguyên nhân thất bại là sự khác biệt giữa bài toán tiền huấn luyện và bài toán Text-to-SQL, nơi lượng dữ liệu huấn luyện hạn chế khiến mô hình ngôn ngữ tiền huấn luyện không thể thích nghi tốt với bộ dữ liệu mới [1].

This paper was submitted for review in November 2025.

All authors are with the Faculty of Information Technology, Industrial University of Ho Chi Minh City (IUH), Vietnam. ThS. Tran Thi Anh Thi is a lecturer of the Software Engineering Faculty Team.

*Corresponding author: tranthianhthi@iuh.edu.vn

Thêm vào đó, để sinh SQL chính xác, Text-to-SQL cần nắm rõ schema (tên bảng, tên cột, quan hệ). Mô hình hóa schema cực khó vì schema thường lớn, phức tạp và khác biệt theo từng domain. Text-to-SQL thường thất bại khi “generalize” giữa các domain — mô hình train cho domain này đem qua domain khác là dễ lỗi, do khác biệt từ schema đến ngôn ngữ tự nhiên [2].

Để giải quyết các thách thức trên, một cách tiếp cận phổ biến là sử dụng mô hình ngôn ngữ lớn (LLM) kết hợp với system prompt chứa schema. Tuy nhiên, các phương pháp fine-tuning Text-to-SQL hiện tại thường đưa tất cả bảng và cột vào prompt. Điều này có hai vấn đề chính: Đầu tiên, khi cơ sở dữ liệu lớn, mô hình khó xử lý ngữ cảnh dài như vậy. Thứ hai, không phải bảng/cột nào cũng liên quan đến câu hỏi, nên đưa thừa thông tin không chỉ tăng chi phí tính toán mà còn làm mô hình mất tập trung vào phần quan trọng, dẫn đến giảm hiệu năng [3]. Hơn nữa, việc nhét toàn bộ schema vào prompt dù giúp tăng độ chính xác nhưng lại làm tăng rủi ro lộ thông tin schema, tiềm ẩn nguy cơ bảo mật nghiêm trọng cho hệ thống cơ sở dữ liệu [4].

Thêm vào đó, trong quá trình inference (Suy luận), các hệ thống Text-to-SQL hiện tại gặp phải vấn đề chưa tối ưu hóa tái sử dụng SQL, gây lãng phí nguồn lực. Lỗi phổ biến ở pha inference là lỗi hệ thống như API fail hoặc prompt quá dài, làm mất kết quả. Việc liên tục nạp cùng một câu hỏi hoặc câu tương đương vào Text-to-SQL dẫn đến lặp lại công sức sinh và thực thi SQL, dù đã từng sinh ra và thực thi kết quả giống hệt trước đó. Các hệ thống hiện tại chưa có chiến lược chuẩn hóa hoặc cache hóa kết quả SQL này [5].

Trong bối cảnh đó, các hệ thống RAG-Text2SQL mang lại khả năng cải thiện đáng kể trong việc xử lý nhiều loại schema và phép hỏi khác nhau mà không cần retrain, vì có thể tùy ý truy xuất tri thức bổ sung theo từng câu hỏi [5]. Điều này giải quyết được các vấn đề về chi phí huấn luyện lại, khả năng mở rộng và tính bảo mật của schema. Hơn nữa, kết hợp RAG giúp thử nghiệm và xây dựng hệ ứng dụng Text-to-SQL mới cực nhanh, dễ đưa logic nghiệp vụ vào mà không cần tốn nhiều công làm bộ annotate dữ liệu thủ công [6].

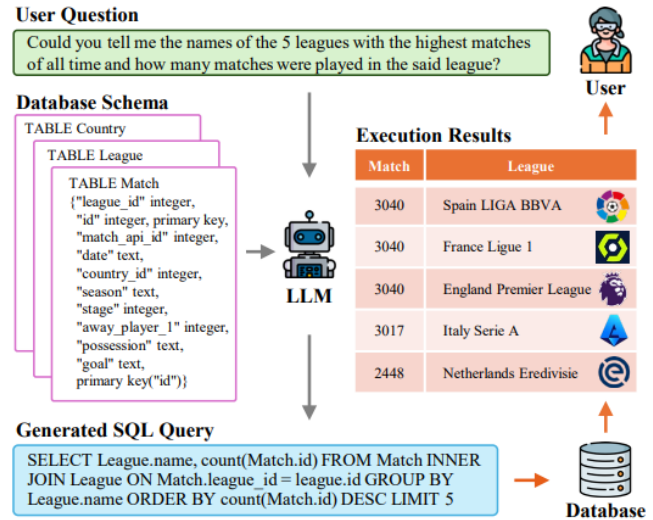
Trustay-AI, một kiến trúc multi-agent tự xác thực cho bài toán Text-to-SQL tăng cường truy xuất (RAG), kế thừa và phát triển các ý tưởng của RAG-Text2SQL và mô hình ngôn ngữ lớn (LLM). Khác với các phương pháp truyền thống yêu cầu huấn luyện trên bộ dữ liệu lớn, Trustay-AI không cần phải có một bộ dữ liệu lớn để train từ đầu. Thay vào đó, hệ thống sử dụng LLM được tiền huấn luyện sẵn kết hợp với cơ chế RAG để truy xuất tri thức nghiệp vụ và schema một cách động theo từng câu hỏi. Tích hợp kiến trúc 4 agent tuần tự (Orchestrator Agent, SQL Generation Agent, Response Generator Agent, Result Validator Agent) tự động hóa quá trình hiểu ý định người dùng, sinh truy vấn SQL tối ưu, định dạng kết quả và kiểm định sự khớp ngữ nghĩa. Đặc biệt, Trustay-AI trang bị hai cơ chế then chốt: (1) Cơ chế tự học (self-learning), trong đó Result Validator Agent đánh giá và lưu trữ các cặp câu hỏi-SQL đã được xác thực vào kho tri thức, giúp hệ thống cải thiện độ chính xác theo thời gian mà không cần huấn luyện lại; (2) Cơ chế tái sử dụng SQL canonical (canonical SQL

reuse), trong đó hệ thống kiểm tra độ tương đồng ngữ nghĩa giữa câu hỏi hiện tại và các câu hỏi đã lưu, cho phép truy xuất và tái sử dụng SQL đã được xác thực thay vì phải generate lại, giúp giảm chi phí tính toán và tăng tốc độ phản hồi. Nhờ đó, Trustay-AI hứa hẹn tăng độ chính xác, giảm chi phí phát triển, tối ưu hóa tài nguyên tính toán, và định hình tương lai của các hệ thống truy cập dữ liệu hội thoại.

II. CƠ SỞ LÝ THUYẾT

A. Text-to-SQL

Text-to-SQL là bài toán chuyển đổi truy vấn ngôn ngữ tự nhiên (Natural Language Query – NLQ) thành câu lệnh SQL có cấu trúc, giúp người dùng truy cập cơ sở dữ liệu mà không cần biết cú pháp SQL. Các mô hình ban đầu như Seq2SQL và SQLNet áp dụng cơ chế học sâu để học ánh xạ giữa câu hỏi và câu lệnh SQL, trong khi các mô hình hiện đại như T5-SQL, PICARD và GPT-SQL dựa trên mô hình ngôn ngữ lớn (LLM) nhằm cải thiện khả năng hiểu ngữ cảnh và sinh cú pháp phức tạp. Tuy nhiên, các mô hình này vẫn gặp hạn chế về độ chính xác ngữ nghĩa, thiếu tri thức nghiệp vụ (business context) và không có cơ chế xác thực hậu kiểm. Khi mô hình LLM sinh SQL, lỗi về điều kiện lọc, ngữ cảnh hoặc cấu trúc dữ liệu vẫn có thể xảy ra. Do đó, các hướng tiếp cận gần đây tập trung vào việc kết hợp truy xuất tri thức và kiểm định kết quả, tạo nên cơ sở cho kiến trúc Trustay-AI.



Hình 1. Quy trình cơ bản của hệ thống LLM-based Text-to-SQL: người dùng đưa ra câu hỏi ngôn ngữ tự nhiên, hệ thống cung cấp schema và câu hỏi cho LLM, LLM sinh câu lệnh SQL, SQL được thực thi trên database và trả kết quả về cho người dùng [2].

B. Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) là kỹ thuật kết hợp giữa truy xuất thông tin (retrieval) và sinh ngôn ngữ (generation) nhằm cải thiện độ chính xác và khả năng cập nhật của các mô hình ngôn ngữ lớn. Trong RAG, truy vấn đầu vào được mã hóa thành vector ngữ nghĩa, sau đó hệ thống truy tìm các đoạn văn hoặc tài liệu có liên quan trong kho tri

thức vector bằng độ tương đồng cosine, được tính theo công thức:

$$\text{sim}(x, y) = \frac{x \cdot y}{\|x\| \|y\|} \quad (1)$$

Trong đó, x và y là hai vector biểu diễn ngữ nghĩa của truy vấn và tài liệu, ký hiệu “.” là tích vô hướng giữa hai vector, còn $\|x\|$ và $\|y\|$ là chuẩn Euclid (độ dài) của mỗi vector [7]–[9]. Giá trị $\text{sim}(x, y)$ nằm trong khoảng từ 0 đến 1, càng gần 1 thì mức độ tương đồng càng cao. Các đoạn tri thức có giá trị tương đồng lớn nhất sẽ được chọn làm ngữ cảnh, sau đó được ghép vào prompt đầu vào của LLM để sinh câu trả lời hoặc câu lệnh SQL.

1) *Pipeline RAG trong Text-to-SQL*: Công thức tổng quát cho RAG trong Text2SQL có thể được mô tả qua các bước trong pipeline chuẩn. Đầu tiên, truy vấn đầu vào (q) là câu hỏi ngôn ngữ tự nhiên của người dùng liên quan đến dữ liệu. Tiếp theo, truy xuất tri thức ($D_{\text{retrieve}}(q)$) sử dụng module truy xuất (retriever, ví dụ BM25, Dense vector, Hybrid search) để tìm ra tập hợp đoạn/tài liệu/schema liên quan nhất R_q từ cơ sở tri thức hoặc metadata (D):

$$R_q = D_{\text{retrieve}}(q) \quad (2)$$

Sau đó, ghép nối ngữ cảnh (context fusion) kết hợp truy vấn với các đoạn tri thức vừa truy xuất thành một prompt tổng hợp:

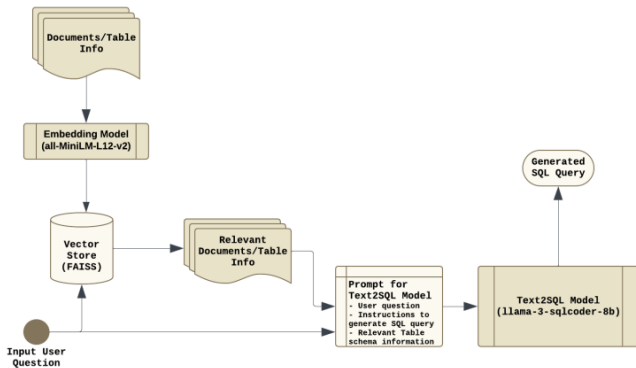
$$\text{prompt} = \text{concat}(q, R_q) \quad (3)$$

Tiếp theo, sinh câu trả lời bằng LLM (y) đưa prompt đã bổ sung ngữ cảnh vào LLM, sinh ra câu trả lời đầy đủ (ở đây là câu SQL chuẩn hóa):

$$y = \text{LLM}(\text{prompt}) \quad (4)$$

Cuối cùng, (Tuỳ chọn) Kiểm định hậu kiểm (Verifier) áp dụng module kiểm định hoặc execution check sau khi sinh SQL [5]. Công thức minh họa đơn giản của toàn bộ pipeline có thể được viết gọn như sau:

$$y = \text{LLM}(\text{concat}(q, D_{\text{retrieve}}(q))) \quad (5)$$



Hình 2. Quy trình pipeline RAG-Text2SQL: User Query → Retriever (truy xuất schema/document/vector) → Fusion Context → LLM (Text2SQL Generator) → SQL Output [10].

Quy trình RAG kết hợp truy xuất tri thức phù hợp với truy vấn đầu vào, sau đó nối thông tin ngữ cảnh vào prompt của LLM để tạo ra trả lời chính xác và cập nhật mới nhất [5], [9], [11]. Cách này giúp hạn chế ảo giác, tăng độ nhất quán và hiệu quả phản hồi. Trong Trustay-AI, RAG đóng vai trò cung cấp ngữ cảnh nghiệp vụ cho Orchestrator Agent, giúp mô hình sinh SQL hiểu được cấu trúc dữ liệu và mối quan hệ giữa các thực thể trong cơ sở dữ liệu thực tế. Công thức trên áp dụng chuẩn cho mọi ứng dụng Text2SQL hiện đại dùng RAG, giúp cải thiện cả độ chính xác lẫn khả năng thích nghi với domain mới, schema lớn.

C. Hệ thống Multi-Agent

Hệ thống Multi-Agent gồm nhiều thực thể trí tuệ nhân tạo (Agent) có khả năng tự động xử lý, giao tiếp và phối hợp để hoàn thành một nhiệm vụ phức tạp. Các thí nghiệm cho thấy, so với cấu hình một agent, framework multi-agent LLM tăng đáng kể tỷ lệ thành công và sự đa dạng giải pháp, đặc biệt ở các domain phức tạp, mở. Việc phối hợp giữa các agent chuyên biệt mang lại khả năng giải quyết vấn đề vững chắc và sáng tạo hơn [12], [13]. Trong lĩnh vực xử lý ngôn ngữ tự nhiên, mô hình đa agent cho phép chia nhỏ nhiệm vụ và thực hiện song song các bước như phân tích ngữ nghĩa, sinh truy vấn, xác thực kết quả, giúp nâng cao độ chính xác tổng thể. Các framework như AutoGen, CAMEL và ReAct đã chứng minh tính hiệu quả của cách tiếp cận này khi nhiều Agent phối hợp trên cùng một tác vụ sinh – kiểm – sửa (generate–verify–refine). Trustay-AI áp dụng cấu trúc bốn Agent hoạt động tuần tự và có cơ chế phản hồi liên tục, trong đó các Agent vừa hỗ trợ vừa kiểm soát lẫn nhau để đảm bảo tính chính xác của toàn bộ pipeline.

D. Canonical SQL Reuse và Validation Logic

Cơ chế Canonical SQL Reuse dựa trên nguyên lý retrieval-based approach, trong đó hệ thống truy xuất và tái sử dụng các câu lệnh SQL đã được sinh và xác thực trước đó. Theo nghiên cứu về các framework retrieval-based cho Text-to-SQL, việc chọn các few-shot examples với SQL skeleton tương tự cho các câu hỏi tương đương giúp cung cấp hướng dẫn cú pháp, từ đó sinh ra các kết quả SQL chính xác hơn về mặt cú pháp. Cách tiếp cận này nâng cao quá trình học trong ngữ cảnh (in-context learning) của LLM [11]. Cơ chế Canonical SQL Reuse cho phép hệ thống tái sử dụng các mẫu câu lệnh SQL đã được xác thực nhằm tăng hiệu quả xử lý và giảm rủi ro sai cú pháp. Khi người dùng gửi truy vấn mới, hệ thống tính độ tương đồng ngữ nghĩa giữa truy vấn hiện tại (Q) và các truy vấn đã lưu (Q_i) bằng công thức:

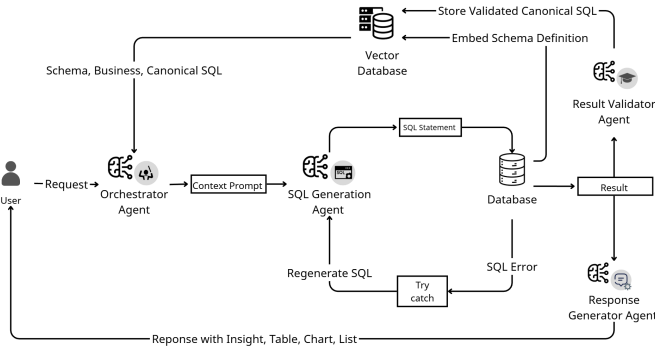
$$\text{reuse}(Q, Q_i) = \text{sim}(E_Q, E_{Q_i}) \quad (6)$$

Trong đó, E_Q và E_{Q_i} là vector embedding của truy vấn đầu vào và truy vấn đã có trong kho tri thức, còn $\text{sim}(\cdot, \cdot)$ là hàm tính độ tương đồng cosine như đã mô tả ở trên [5], [11]. Nếu $\text{reuse}(Q, Q_i)$ vượt ngưỡng định sẵn ($\tau_{\text{hard}} = 0.92$ hoặc $\tau_{\text{soft}} = 0.80$), hệ thống sẽ sử dụng lại hoặc tinh chỉnh câu lệnh SQL tương ứng.

Song song đó, Trustay-AI áp dụng **cơ chế xác thực định tính dựa trên LLM** để đánh giá độ khớp giữa yêu cầu đầu vào và kết quả sinh ra. Thay vì tính điểm định lượng, hệ thống cung cấp cho *Result Validator Agent* ba yếu tố (truy vấn, SQL đã sinh, và bản xem trước kết quả), từ đó Agent trả về phán quyết nhị phân `IS_VALID: true/false`. Chỉ các kết quả được đánh giá hợp lệ mới được lưu vào kho tri thức [2], [5].

III. KIẾN TRÚC HỆ THỐNG TRUSTAY-AI

A. Tổng quan kiến trúc



Hình 3. Luồng xử lý của Trustay-AI, bao gồm RAG, tự sửa lỗi (Regenerate SQL) và tự xác thực (Validator).

Trustay-AI được thiết kế dưới dạng một kiến trúc multi-agent tự xác thực, gồm bốn Agent hoạt động tuần tự với các vòng phản hồi tự điều chỉnh. Luồng xử lý bắt đầu khi người dùng gửi truy vấn ngôn ngữ tự nhiên vào hệ thống.

Orchestrator Agent tiếp nhận truy vấn này và truy xuất ngữ cảnh nghiệp vụ từ Vector Database, bao gồm schema của các bảng liên quan, các câu lệnh SQL đã được xác thực trước đó (Canonical SQL), và ngữ cảnh ứng dụng. Sau đó, Agent này phân tích ý định người dùng, xác định vai trò trước khi chuyển tiếp toàn bộ thông tin đã được làm giàu ngữ cảnh cho SQL Generation Agent. Nếu hệ thống không đủ thông tin, Orchestrator Agent có thể yêu cầu người dùng làm rõ (clarification) trước khi chuyển tiếp.

SQL Generation Agent nhận thông tin từ Orchestrator Agent, kiểm tra khả năng tái sử dụng mẫu canonical SQL, và sinh câu lệnh SQL tối ưu. Câu lệnh SQL được gửi đến Database để thực thi, với Schema Definition được cung cấp từ Vector Database. Nếu quá trình thực thi gặp lỗi (SQL Error), cơ chế Try-catch sẽ phát hiện và yêu cầu SQL Generation Agent tái sinh câu lệnh SQL (Regenerate SQL), tạo thành vòng phản hồi tự sửa lỗi.

Khi SQL được thực thi thành công, Database trả về Result. Kết quả này được chia thành hai nhánh xử lý song song: **Response Generator Agent** biến đổi Result thành phản hồi có cấu trúc (Insight, Table, Chart, hoặc List) và trả về cho người dùng; đồng thời, **Result Validator Agent** đánh giá tính hợp lệ của kết quả bằng cách so sánh câu hỏi đầu vào, câu lệnh SQL đã sinh và bản xem trước kết quả (results preview) để đưa ra **phán quyết logic** dạng `IS_VALID: true/false`.

Chỉ khi kết quả được xác nhận hợp lệ, Result Validator Agent mới lưu Knowledge SQL vào Vector Database.

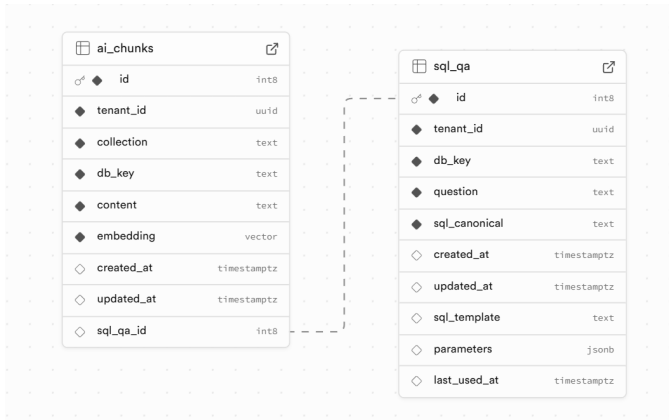
Vector Database đóng vai trò trung tâm trong kiến trúc này. Nó vừa cung cấp Schema Definition cho Database để đảm bảo ngữ cảnh thực thi chính xác, vừa cung cấp Schema, Context và Knowledge SQL cho Orchestrator Agent để làm giàu ngữ cảnh truy vấn. Đồng thời, Vector Database nhận tri thức mới từ Result Validator Agent, tạo thành vòng học tự động (self-learning loop) giúp hệ thống cải thiện độ chính xác theo thời gian.

Một điểm đặc biệt trong luồng xử lý của Trustay-AI là cơ chế tối ưu hóa thông minh được tích hợp ở nhiều tầng. Trước khi SQL Generation Agent sinh câu lệnh mới, hệ thống sẽ kiểm tra khả năng tái sử dụng các canonical SQL đã được lưu trữ bằng cách tính độ tương đồng ngữ nghĩa giữa truy vấn hiện tại và các truy vấn trong kho tri thức. Nếu độ tương đồng vượt ngưỡng (0.85–0.92), hệ thống sẽ ưu tiên sử dụng hoặc tinh chỉnh SQL template đã có thay vì sinh mới hoàn toàn, giúp giảm đáng kể thời gian xử lý và chi phí tính toán. Đồng thời, Response Generator Agent không chỉ đơn thuần chuyển đổi kết quả SQL thành văn bản, mà còn phân tích ngữ cảnh của truy vấn để quyết định định dạng phù hợp nhất (LIST, TABLE, CHART, hoặc INSIGHT), đảm bảo người dùng nhận được thông tin được trình bày một cách trực quan và dễ hiểu nhất. Cơ chế xử lý song song giữa Response Generator và Validator Agent cho phép hệ thống vừa trả kết quả nhanh chóng cho người dùng, vừa thực hiện kiểm định và học hỏi ở background, tối ưu hóa trải nghiệm người dùng mà không làm chậm quá trình tích lũy tri thức.

B. Kiến trúc Vector Database và RAG Ingestion

Hệ thống tri thức (knowledge base) của Trustay-AI được xây dựng trên một cơ sở dữ liệu vector, triển khai trên PostgreSQL sử dụng extension `pgvector`. Kiến trúc này cho phép truy xuất tri thức theo ngữ cảnh một cách hiệu quả và hỗ trợ đa người dùng (multi-tenant) thông qua phân tách logic `tenant_id`.

Tri thức được tổ chức trong hai bảng chính: (1) **sql_qa**, lưu trữ các canonical SQL đã được xác thực để tái sử dụng, và (2) **ai_chunks**. Bảng `ai_chunks` lưu trữ các vector embeddings (sử dụng mô hình Google GenAI `text-embedding-004` với 768-dimension) và được phân chia thành bốn “collections” logic: `schema`, `qa` (dùng chung với bảng `sql_qa`), `business` (lưu quy tắc nghiệp vụ), và `docs` (lưu tài liệu đã denormalize).



Hình 4. Kiến trúc schema của Vector Database: Bảng ai_chunks lưu trữ vector embeddings và nội dung, liên kết với bảng sql_qa thông qua foreign key sql_qa_id. Bảng sql_qa lưu trữ các cặp câu hỏi-SQL đã được xác thực dưới dạng canonical SQL và template tham số hóa.

Để giải quyết vấn đề phân mảnh ngữ cảnh (context fragmentation) thường thấy trong các hệ thống RAG, chúng tôi áp dụng một chiến lược ingestion gọi là `table_complete`. Thay vì chia nhỏ từng cột hay từng quan hệ, chiến lược này biểu diễn mỗi bảng trong database thành một chunk JSON duy nhất, được lưu vào ai_chunks như một đơn vị tri thức nguyên khối và có thể được truy xuất, cache lại nhiều lần. Về cấu trúc, mỗi chunk `table_complete` bắt đầu với `"chunk_type": "table_complete"`, `"table_name"` (ví dụ `"users"`), `"database_name"` (ví dụ `"trustay_core"`) và `"business_context"` mô tả ngắn gọn vai trò nghiệp vụ (hệ người dùng, tenant/landlord, đăng nhập). Kế tiếp là `"constraints"` ở mức bảng (khóa chính `users_pkey` trên `id`), mảng `"columns"` với các cột chính (`id`, `email`, `role`, `overall_rating`) kèm kiểu dữ liệu, ràng buộc (`NOT NULL`, `DEFAULT 0`, enum `"tenant"/"landlord"`) và `sample_values`, một bản ghi mẫu trong `"sample_rows"`, cùng vài câu SQL điển hình trong `"sample_queries"`. Cách biểu diễn này gom toàn bộ thông tin schema và ví dụ sử dụng vào một khối duy nhất, giúp giảm số mảnh phải ghép trong prompt, thu nhỏ kích thước context gửi vào LLM và hạn chế nguy cơ mất ngữ cảnh khi sinh SQL.

Ví dụ cụ thể về cấu trúc JSON của một chunk `table_complete` cho bảng người dùng `users` trong database `trustay_core` được minh họa ở Hình 5. Hình này cho thấy rõ các nhóm trường: phần header với các khóa `chunk_type`, `table_name`, `database_name`, `business_context`; phần `constraints` khai báo khóa chính `users_pkey` trên cột `id`; và danh sách `columns` mô tả các cột `id` (kiểu `TEXT`, ràng buộc `NOT NULL`, ví dụ một `UUID`), `email` (kiểu `TEXT`, `NOT NULL`, ví dụ một địa chỉ email thật), `role` (kiểu `ENUM` với hai giá trị `tenant/landlord`, có `NOT NULL` và `DEFAULT tenant`) và `overall_rating` (kiểu `NUMERIC` với `DEFAULT 0`). Bên dưới, phần `sample_rows` chứa một bản ghi mẫu (bao gồm `id`, `email`, `role` và `overall_rating`), trong khi `sample_queries` liệt kê các truy vấn SQL cơ bản trên bảng `users` (ví dụ truy vấn toàn

bộ bảng hoặc đếm số dòng). Khi Orchestrator Agent và SQL Generation Agent truy xuất chunk này, chúng nhận được một ngữ cảnh nhất quán và giàu ví dụ về thực thể `users`, từ đó dễ suy luận vai trò của từng cột trong các quy trình nghiệp vụ và giảm đáng kể sự mơ hồ khi sinh SQL.

table_complete cho bảng users	JSON
	<pre> { "chunk_type": "table_complete", "table_name": "users", "database_name": "trustay_core", "business_context": "Users - system users, tenant/landlord, login.", "constraints": [{ "name": "users_pkey", "type": "PRIMARY KEY", "columns": ["id"] }], "columns": [{ "name": "id", "type": "TEXT", "constraints": ["NOT NULL"], "sample_values": ["0f58350c-fd08-4890-968d-acf5a410740a"] }, { "name": "email", "type": "TEXT", "constraints": ["NOT NULL"], "sample_values": ["budget.student@trustay.com"] }, { "name": "role", "type": "ENUM(tenant landlord)", "constraints": ["NOT NULL", "DEFAULT 'tenant'"], "enum_values": ["tenant", "landlord"] }, { "name": "overall_rating", "type": "NUMERIC", "constraints": ["DEFAULT 0"], "sample_values": ["0"] }], "sample_rows": [{ "id": "1474463c-de57-4be0-a51c-f7805db8a423", "email": "budget.student@trustay.com", "role": "tenant", "overall_rating": "0" }], "sample_queries": ["SELECT * FROM users LIMIT 10", "SELECT COUNT(*) FROM users"] }</pre>

Hình 5. Ví dụ phần content của ai_chunks với nội dung là JSON `table_complete` cho bảng `users` trong database `trustay_core`, minh họa đầy đủ các trường `business_context`, `constraints`, `columns`, `sample_rows` và `sample_queries` mà LLM nhận được.

Cách tiếp cận này đảm bảo rằng khi một truy vấn được thực hiện, SQL Generation Agent luôn nhận được ngữ cảnh đầy đủ và không mơ hồ của một thực thể, giúp giảm đáng kể lỗi sinh SQL do thiếu thông tin. Quá trình truy xuất (retrieval) sử dụng cosine distance (hàm `match_ai_chunks`) với các ngưỡng (thresholds) và giới hạn (dynamic limits) được tùy chỉnh. Đặc biệt, các agent khác nhau sẽ truy vấn các collection khác nhau:

Orchestrator Agent truy vấn `business` để hiểu ý định, trong khi SQL Generation Agent truy vấn `schema` để lấy cấu trúc, đảm bảo tính chuyên biệt của ngữ cảnh.

C. Mô hình xác thực và học tự động

Một đóng góp cốt lõi của Trustay-AI là cơ chế xác thực và học hỏi liên tục. Sau khi SQL Generation Agent trả về kết quả, **Result Validator Agent** sẽ thực hiện một bước kiểm định định tính. Agent này nhận đầu vào là một bộ ba (triad) gồm: (1) truy vấn gốc, (2) câu lệnh SQL đã sinh, và (3) bản xem trước kết quả.

Dựa trên bộ ba này, Agent đưa ra phán quyết logic về tính hợp lệ ngữ nghĩa (`IS_VALID: true/false`) và mức độ nghiêm trọng (`SEVERITY: ERROR/WARN`). Cơ chế này kích hoạt một vòng học tự động (self-learning loop): hệ thống chỉ lưu trữ tri thức mới nếu truy vấn được xác nhận là hợp lệ và không chứa lỗi nghiêm trọng (ví dụ: `isValid = true` và `severity != 'ERROR'`).

Khi điều kiện lưu được thỏa mãn, tri thức được lưu trữ kép: (1) Bảng `sql_qa` lưu trữ canonical SQL dưới dạng template tham số hóa (parameterized template) để tái sử dụng; và (2) Bảng `ai_chunks` (collection `qa`) lưu vector của câu hỏi, liên kết đến `sql_qa_id` để tra cứu. Khi người dùng đặt câu hỏi tương tự trong tương lai, SQL Generation Agent có thể truy xuất các canonical SQL này (với ngưỡng tương đồng 0.85–0.92) để tái sử dụng, giúp tăng tốc độ và độ chính xác theo thời gian.

D. Định dạng phản hồi và trao đổi giữa các Agent

Để giải quyết vấn đề cố hữu của LLM là tính không nhất quán trong định dạng đầu ra, vốn thường gây ra lỗi phân tích cú pháp (parsing errors), hệ thống áp dụng một kiến trúc **“JSON Envelope”** với nguyên tắc phân tách trách nhiệm (Separation of Concerns) rõ ràng.

Cụ thể, **Response Generator Agent** (sử dụng LLM) chỉ chịu trách nhiệm sinh phần văn bản diễn giải thân thiện với người dùng (message). Trong khi đó, toàn bộ cấu trúc dữ liệu (payload) chứa mode (LIST, TABLE, CHART, INSIGHT) và dữ liệu đã được định dạng (`list`, `table`, `chart`) được xây dựng bởi Business Logic Layer.

Phản hồi cuối cùng luôn là một gói JSON hợp lệ 100%:

```
{
  "message": "Đây là kết quả phân tích...",
  "payload": {
    "mode": "CHART",
    "chart": { "type": "bar", "data": [...] },
    "meta": { "sql_query": "SELECT..." }
  }
}
```

Cách tiếp cận này đảm bảo client luôn nhận được dữ liệu có cấu trúc đáng tin cậy, loại bỏ hoàn toàn lỗi parsing, đồng thời giải phóng LLM để tập trung vào thế mạnh cốt lõi là sinh ngôn ngữ chất lượng cao.

IV. ĐÓNG GÓP KỸ THUẬT VÀ ỨNG DỤNG

Kiến trúc mô-đun và tự xác thực của Trustay-AI mang lại hai đóng góp và ứng dụng thực tế rõ rệt, giải quyết các vấn đề cốt lõi được đặt ra trong phần Giới thiệu.

A. Giải quyết Nghịch lý “Khởi động lạnh” (Zero-to-Golden)

Đây là một ứng dụng có tính mới, liên kết trực tiếp với tiêu đề “From Zero to Golden” của bài báo. Vấn đề lớn nhất của Text-to-SQL là chi phí “chú thích” (annotate) thủ công hàng ngàn cặp (Câu hỏi, SQL) để tạo “dữ liệu vàng” (golden data) cho việc huấn luyện.

Kiến trúc của Trustay-AI được thiết kế để giải quyết vấn đề này bằng cách hoạt động như một hệ thống “thu hoạch và xác thực” tri thức. Thay vì yêu cầu dữ liệu trước, hệ thống học hỏi trong quá trình vận hành. Mỗi khi một cặp (Câu hỏi, SQL) được Result Validator Agent đánh giá là hợp lệ (`IS_VALID = true` và `severity != 'ERROR'`), nó sẽ được lưu trữ vào kho tri thức (Bảng `sql_qa` và `ai_chunks`).

Kho tri thức “canonical SQL” này chính là một bộ “dữ liệu vàng” (dataset) “sạch”, chất lượng cao, đã được xác minh về mặt ngữ nghĩa. Nó sẵn sàng được sử dụng để fine-tune các mô hình Text-to-SQL thế hệ tiếp theo, giúp giảm đáng kể chi phí nhân công và giải quyết triệt để vấn đề “khởi động lạnh”.

B. Hệ thống Phân tích Dữ liệu (BI) Tương tác

Đây là ứng dụng cốt lõi của hệ thống. Thay vì chỉ trả về dữ liệu thô, kiến trúc của Trustay-AI cho phép nó vượt xa việc truy vấn đơn thuần. Bằng cách tận dụng Response Generator Agent và Business Logic Layer, hệ thống được thiết kế để tự động chuyển đổi kết quả SQL thành các định dạng trực quan (LIST, TABLE, CHART, INSIGHT) phù hợp với ngữ cảnh phân tích (BI).

Khả năng này cho phép người dùng quản lý đặt các câu hỏi phân tích phức tạp, ví dụ như “So sánh tỷ lệ lấp đầy giữa hai tòa nhà trong 6 tháng qua?”, và nhận về một biểu đồ thay vì một bảng dữ liệu thô.

Ngoài ra, cơ chế Canonical SQL Reuse được thiết kế để đặc biệt hiệu quả trong môi trường BI, nơi các truy vấn trên dashboard (ví dụ: “doanh thu tháng này”) thường được lặp lại. Bằng cách lưu và tái sử dụng các SQL đã được xác thực, hệ thống có thể giảm đáng kể độ trễ và chi phí tính toán cho các tác vụ lặp lại.

V. KẾT QUẢ

A. Thiết lập thí nghiệm

Để đánh giá hiệu quả thực tế của Trustay-AI, chúng tôi đã xây dựng một bộ 20 test case được thiết kế để mô phỏng các truy vấn thực tế của người dùng. Các truy vấn này đòi hỏi các thông tin chi tiết có thể hành động được (actionable insights), ví dụ như “Liệt kê danh sách hợp đồng sắp hết hạn trong 30 ngày tới” hoặc “Tính doanh thu trung bình mỗi phòng cho Tòa nhà A trong ba tháng qua.” Bộ truy vấn được cấu trúc thành bốn nhóm thách thức riêng biệt: (i) truy vấn đơn giản yêu cầu ngữ cảnh ngầm; (ii) truy vấn logic nghiệp vụ phức tạp liên quan đến JOIN đa bảng; (iii) truy vấn nhập nhằng đòi hỏi hiểu

biết ngữ nghĩa domain; và (iv) kiểm tra độ bền (robustness) với các từ đồng nghĩa và lỗi chính tả.

Chúng tôi so sánh hai cấu hình dựa trên cùng một mô hình ngôn ngữ lớn Google Gemini Flash 2.0¹:

- **Mô hình A (Baseline):** Gemini Flash 2.0 được cung cấp một system prompt tính chứa toàn bộ Database schema và hướng dẫn chung “hãy sinh SQL chính xác”. Mọi truy vấn được gửi trực tiếp vào mô hình; không có bước lập kế hoạch, truy xuất tri thức hay kiểm định.
- **Mô hình B (Trustay-AI):** Kiến trúc Multi-Agent + RAG của chúng tôi gồm Orchestrator Agent, SQL Generation Agent, Response Generator Agent và Result Validator Agent. Tất cả agent đều sử dụng cùng một backbone Gemini Flash 2.0 nhưng phối hợp qua các thông điệp JSON có cấu trúc để phân rõ nhiệm vụ, truy xuất tri thức domain, sinh và xác thực SQL theo vòng lặp tự sửa lỗi.

Script đánh giá được triển khai trong ứng dụng quản lý cho thuê Trustay để đo chính xác việc thực thi câu lệnh SQL giữa hai mô hình. Chúng tôi chấm điểm bằng **độ chính xác thực thi** (Execution Accuracy – EX), tức tỷ lệ truy vấn được thực thi thành công và trả về kết quả phù hợp mục tiêu nghiệp vụ sau khi đối chiếu thủ công với ground-truth. Mỗi truy vấn được chạy độc lập nhiều lần để giảm nhiễu; chúng tôi ghi nhận kết quả tốt nhất (best-of-N) và báo cáo độ lệch chuẩn ở mỗi danh mục.

B. Kết quả hiệu suất tổng thể (Overall Performance)

Bảng I cho thấy Trustay-AI vượt trội rõ rệt so với baseline về độ chính xác tổng thể lẫn tính ổn định. Cụ thể:

- **Mô hình A (Baseline)** đạt EX trung bình **45.8%** (tương đương 13.75/30 điểm). Mô hình này thất bại trong việc tạo ra SQL hợp lệ ở 8/20 truy vấn phức tạp (40% số case) và mắc lỗi bảo mật nghiêm trọng khi làm rò rỉ dữ liệu người dùng.
- **Mô hình B (Trustay-AI)** đạt EX trung bình **80.0%** (tương đương 24/30 điểm), với các lỗi còn lại tập trung ở các bài toán logic phức tạp nhất.
- Mức cải thiện tuyệt đối là 34.2%, tương ứng giảm 63% số truy vấn thất bại so với baseline.

Mức chênh lệch này đến từ ba yếu tố cốt lõi của Trustay-AI: (i) khả năng **hiểu ngữ cảnh trang (page context awareness)** mà baseline hoàn toàn bỏ qua; (ii) khả năng **tự sửa lỗi SQL (self-correction)** khi gặp lỗi cú pháp, trong khi baseline bị crash; và (iii) khả năng **áp dụng bảo mật cấp độ người dùng** mà baseline đã vi phạm.

C. Đánh giá hiệu năng cơ chế Canonical SQL Reuse

Ngoài độ chính xác, một lợi ích quan trọng khác của Canonical SQL Reuse là giảm độ trễ phản hồi và chi phí token cho các truy vấn lặp lại. Để định lượng tác động này, chúng tôi đề xuất một bộ thí nghiệm bổ sung so sánh hiệu năng giữa hai lượt suy luận liên tiếp trên cùng một tập câu hỏi: (i) **lượt đầu tiên chưa có canonical SQL mẫu** trong kho

tri thức (hệ thống buộc phải generate SQL mới bằng LLM cho từng truy vấn), và (ii) **lượt thứ hai được phép tái sử dụng canonical SQL** đã được lưu từ lượt đầu (hệ thống ưu tiên truy xuất và reuse thay vì generate lại nếu truy vấn đủ tương tự).

Thiết lập thí nghiệm sử dụng cùng bộ test case trong Mục V-A nhưng tập trung vào một tập con gồm ba kịch bản có tần suất lặp cao trong thực tế: (1) truy vấn thống kê doanh thu định kỳ, (2) truy vấn trạng thái phòng trên dashboard quản lý, và (3) truy vấn báo cáo hóa đơn chưa thanh toán theo người dùng. Mỗi kịch bản được chạy nhiều lần ở cả hai cấu hình, trong đó các tham số hệ thống (mô hình LLM, context RAG, cấu hình hạ tầng) được giữ cố định để đảm bảo tính công bằng.

Chúng tôi đo lường hai chỉ số chính: (i) **thời gian xử lý end-to-end** (tính từ khi nhận NLQ đến khi trả kết quả đã định dạng cho người dùng), và (ii) **số token tiêu tốn** ở phía LLM (tổng input + output) cho toàn bộ pipeline. Kỳ vọng của chúng tôi là cơ chế Canonical SQL Reuse sẽ rút ngắn đáng kể thời gian xử lý ở các lượt truy vấn thứ hai trở đi, đồng thời giảm lượng token cần thiết vì không phải luôn gửi đầy đủ ngữ cảnh schema phức tạp vào LLM.

Kết quả thực nghiệm cho thấy cơ chế Canonical SQL Reuse mang lại cải thiện đáng kể về thời gian xử lý. Trên ba truy vấn khác nhau, chúng tôi quan sát được mức giảm thời gian phản hồi rõ rệt khi hệ thống tái sử dụng canonical SQL so với khi phải generate mới: truy vấn thứ nhất giảm từ 8378ms xuống 5579ms (giảm 33.4%), truy vấn thứ hai giảm từ 6000ms xuống 5000ms (giảm 16.7%), và truy vấn thứ ba giảm từ 9000ms xuống 6000ms (giảm 33.3%). Đáng chú ý, số token tiêu tốn giữa các lượt không thay đổi đáng kể, điều này cho thấy cơ chế reuse không làm giảm chi phí token mà tập trung vào việc tối ưu hóa thời gian phản hồi.

Quan trọng hơn, mặc dù cơ chế Canonical SQL Reuse chủ yếu cải thiện tốc độ xử lý, nó cũng mang lại lợi ích gián tiếp về độ chính xác. Bằng cách nạp các câu SQL đã được xác thực trước đó vào context của LLM, hệ thống có thể tận dụng các mẫu SQL đúng đã được kiểm định, giúp tăng khả năng sinh ra câu SQL chính xác hơn so với việc generate hoàn toàn mới. Điều này đặc biệt có ý nghĩa trong các kịch bản truy vấn lặp lại, nơi mà các mẫu canonical SQL đã được tích lũy từ các lượt trước có thể được tái sử dụng như các few-shot examples chất lượng cao.

D. Phân tích chi tiết kết quả (Detailed Analysis)

Để minh họa cách Trustay-AI khắc phục các điểm yếu của baseline, chúng tôi phân tích các trường hợp điển hình (case studies) trong quá trình thực nghiệm.

Sự vượt trội của Trustay-AI thể hiện rõ ở các truy vấn thống kê phức tạp (ví dụ: "doanh thu 6 tháng"). Model A (Baseline) crash ngay lập tức với lỗi Only SELECT queries allowed, không thể tự phục hồi. Ngược lại, Model B (Trustay-AI) đã tự sửa lỗi cú pháp 2 lần (lỗi 42703 và 42P01) để đưa ra SQL chính xác ở lần thứ 3, cho thấy khả năng tự phục hồi của kiến trúc agent thông qua vòng lặp Validator Agent.

Tương tự, với các truy vấn yêu cầu nhận diện ngữ cảnh (ví dụ: "đánh giá phòng hiện tại" hoặc "thống kê phòng

¹Phiên bản Gemini Flash 2.0 (2024Q4) được xây dựng cho tác vụ tốc độ cao, hỗ trợ ngữ cảnh dài và có khả năng lập luận cơ bản.

Bảng 1
SO SÁNH ĐỘ CHÍNH XÁC THỰC THI THEO NHÓM THỬ THÁCH GIỮA MODEL A (BASELINE) VÀ MODEL B (TRUSTAY-AI)

Nhóm thử thách	Điểm	Baseline (A)	Trustay-AI (B)
Truy vấn đơn giản, ngữ cảnh ngầm	7	4/7 (57%)	7/7 (100%)
Logic nghiệp vụ, chuỗi JOIN phức tạp	8	2/8 (25%)	4/8 (50%)
Nhập nhằng, ngữ nghĩa domain (RAG)	10	4/10 (40%)	10/10 (100%)
Robustness (từ đồng nghĩa, typo)	5	3.75/5 (75%)	5/5 (100%)
Tổng cộng (trên 30 điểm)	30	13.75 (45.8%)	24.0 (80.0%)

này"), Model A crash vì không thể hiểu ngữ cảnh trang (currentPage) và modeHint=INSIGHT. Trong khi đó, Model B đã nhận diện chính xác các tham số ngữ cảnh này để filter SQL phù hợp, chứng minh khả năng nhận thức ngữ cảnh của Orchestrator Agent. Đây là lý do Trustay-AI đạt 100% (10/10 điểm) trong nhóm "Ngữ nghĩa domain và Nhập nhằng", trong khi baseline chỉ đạt 40%.

Ở nhóm "Logic nghiệp vụ phức tạp", Trustay-AI xử lý tốt hơn đáng kể (đạt 4/8 điểm so với 2/8 điểm của baseline), thành công ở các truy vấn GROUP BY phức tạp. Tuy nhiên, cả hai mô hình đều thất bại ở hai bài toán khó nhất: một là bài toán logic 'VÀ' (yêu cầu HAVING COUNT = 6 khi liệt kê nhiều tiện ích) và hai là bài toán bảo mật subquery (quên filter owner_id trong truy vấn con).

Đặc biệt, một điểm nổi bật khác của Trustay-AI là khả năng xử lý bảo mật cấp độ người dùng. Trong các truy vấn yêu cầu lọc dữ liệu theo người dùng (ví dụ: "hóa đơn chưa thanh toán của tôi"), Baseline đã thất bại nghiêm trọng bằng cách tạo ra SQL không có filter tenant_id, dẫn đến rò rỉ dữ liệu của tất cả người dùng. Ngược lại, Trustay-AI đã thành công trong việc áp dụng đúng các quy tắc bảo mật cấp độ cơ bản, chứng minh khả năng nhận thức ngữ cảnh người dùng của Orchestrator Agent.

Trong nhóm "Robustness", Trustay-AI đạt 100% (5/5 điểm), xử lý tốt các từ đồng nghĩa (ví dụ: "ký túc xá") và lỗi chính tả đơn giản nhờ phép ánh xạ synonym trong Orchestrator Agent. Baseline đạt 75% (3.75/5 điểm), thất bại ở một số trường hợp map thực thể phức tạp.

E. Phân tích lỗi (Error Analysis)

Hai thất bại nghiêm trọng nhất của Trustay-AI là lỗi bảo mật subquery và lỗi logic 'VÀ-HOẶC'. Cả hai đều là những thất bại về logic phức tạp, không phải lỗi cú pháp.

- Trong trường hợp lỗi bảo mật, SQL Agent đã thất bại trong việc áp dụng filter owner_id vào bên trong một subquery, tạo ra lỗ hổng rò rỉ dữ liệu.
- Trong trường hợp lỗi logic 'VÀ', RAG và SQL Agent đã không nhận diện được ý định 'VÀ' khi người dùng liệt kê nhiều tiện ích, dẫn đến việc sử dụng IN (logic 'HOẶC').

Những quan sát này cho thấy vòng xác thực (Validator Agent) hiện tại của chúng tôi mạnh về cú pháp nhưng cần được tăng cường khả năng kiểm định logic nghiệp vụ và bảo mật (business & security logic validation).

F. Thu thập Bộ Canonical SQL (Canonical SQL Repository)

Ngoài việc đạt độ chính xác cao hơn đáng kể so với baseline, một giá trị quan trọng khác mà Trustay-AI mang lại là khả năng tự động thu thập và tích lũy một bộ "dữ liệu vàng" (golden dataset) trong quá trình vận hành. Mỗi khi Result Validator Agent xác nhận một cặp (Câu hỏi, SQL) là hợp lệ (IS_VALID = true), hệ thống tự động lưu trữ vào bảng sql_qa dưới dạng canonical SQL và template tham số hóa (parameterized template) để tái sử dụng.

Hình 6 trình bày một số ví dụ tiêu biểu từ bộ canonical SQL đã được thu thập trong quá trình thực nghiệm. Mỗi mục trong bảng bao gồm: (1) **question** – câu hỏi ngôn ngữ tự nhiên gốc của người dùng; (2) **sql_canonical** – câu lệnh SQL đã được xác thực và chuẩn hóa.

Bộ canonical SQL này không chỉ giúp hệ thống tăng tốc độ phản hồi cho các truy vấn tương tự trong tương lai (thông qua cơ chế Canonical SQL Reuse), mà quan trọng hơn, nó chính là tài sản dữ liệu quý giá mà doanh nghiệp có thể sử dụng để fine-tune các mô hình Text-to-SQL chuyên sâu, giải quyết triệt để bài toán "khởi động lạnh" (cold start) mà không cần phải đầu tư chi phí nhân công để annotate dữ liệu thủ công.

question text	sql_canonical text
Phòng nào rẻ nhất?	SELECT r.id, r.name, rp.base_price_monthly FROM rooms r JOIN room_pricing rp ON r.id = rp.room_id ORDER BY rp.base_price_monthly
Tìm phòng ở Q1 có đầy đủ đồ	SELECT r.id, r.name AS title, b.name AS building_name, rp.base_price_monthly, 'room' AS entity FROM rooms r JOIN buildings b ON b.id = r.building_id WHERE r.district = 'Q1'
Tìm trọ ở Gò Vấp có máy lạnh	SELECT r.id, r.name AS title, b.name AS building_name, rp.base_price_monthly, 'room' AS entity FROM rooms r JOIN buildings b ON b.id = r.building_id WHERE r.district = 'Gò Vấp'
Phòng nào có đầy đủ nội thất?	SELECT r.id, r.name, r.description FROM rooms r JOIN room_amenities ra ON r.id = ra.room_id JOIN amenities a ON ra.amenity_id = a.id WHERE a.name = 'Full Furnished'
Tìm studio ở Quận 1	SELECT r.id, r.name AS title, b.name AS building_name, rp.base_price_monthly, 'room' AS entity FROM rooms r JOIN buildings b ON b.id = r.building_id WHERE r.district = 'Quận 1' AND r.type = 'studio'
Tìm phòng có gác lửng bán công máy lạnh	SELECT r.id, r.name AS title, b.name AS building_name, rp.base_price_monthly, 'room' AS entity FROM rooms r JOIN buildings b ON b.id = r.building_id WHERE r.district = 'Quận 1' AND r.type = 'studio' AND r.has_loft = true
Thống kê số lượng phòng theo trạng thái	SELECT d.district_name AS label, r.status AS status, COUNT(r.id) AS value FROM room_instances ri JOIN rooms r ON ri.room_id = r.id WHERE ri.status = 'available'
Tôi còn bao nhiêu hóa đơn chưa thanh toán?	SELECT COUNT(*) FROM bills WHERE status = 'pending'
Phòng nào có hợp đồng sắp hết hạn trong tháng này?	SELECT r.id, r.name AS title, b.name AS building_name, rp.base_price_monthly, 'room' AS entity FROM rooms r JOIN buildings b ON b.id = r.building_id WHERE r.district = 'Quận 1'
Doanh thu 6 tháng qua	SELECT SUM(payments.amount) AS total_revenue FROM payments JOIN rentals ON payments.rental_id = rentals.id JOIN room_instances ri ON rentals.room_id = ri.id WHERE rentals.start_date >= DATE_SUB(CURDATE(), INTERVAL 6 MONTH)
Hóa đơn tháng này của tôi	SELECT b.id AS bill_id, b.created_at AS bill_date, b.total_amount, b.status AS bill_status, c.id AS rental_id, p.payment_date, p.amount AS payment_amount FROM bills b JOIN rentals r ON b.rental_id = r.id JOIN payments p ON b.id = p.bill_id WHERE r.start_date >= DATE_SUB(CURDATE(), INTERVAL 1 MONTH)
Phòng nào có máy lạnh và wifi?	SELECT r.id, r.name FROM rooms r JOIN room_amenities ra ON r.id = ra.room_id JOIN amenities a ON ra.amenity_id = a.id WHERE a.name = 'Air Conditioning' AND a.name = 'WiFi'
Có phòng nào dưới 5 triệu không?	SELECT r.id, r.name AS title, b.name AS building_name, rp.base_price_monthly, 'room' AS entity FROM rooms r JOIN buildings b ON b.id = r.building_id WHERE rp.base_price_monthly < 5000000
Tìm phòng trọ ở Gò Vấp	SELECT r.id, r.name AS title, b.name AS building_name, rp.base_price_monthly, 'room' AS entity FROM rooms r JOIN buildings b ON b.id = r.building_id WHERE r.district = 'Gò Vấp' AND r.type = 'studio'
Đánh giá phòng này	SELECT r.id, r.name, r.description, r.slug, r.room_type, r.area_sqm, r.max_occupancy, r.total_rooms, b.name AS building_name, b.address FROM rooms r JOIN buildings b ON r.building_id = b.id
Giá phòng hiện tại có hợp lý không?	SELECT r.id, r.name, r.description, r.slug, r.room_type, r.area_sqm, r.max_occupancy, r.total_rooms, b.name AS building_name, b.address FROM rooms r JOIN buildings b ON r.building_id = b.id

Hình 6. Một số ví dụ tiêu biểu từ bộ canonical SQL đã được thu thập tự động trong quá trình vận hành Trustay-AI. Bảng này minh họa cách hệ thống lưu trữ các cặp (Câu hỏi, SQL Canonical) đã được xác thực, tạo thành một kho tri thức có thể tái sử dụng và phục vụ cho việc fine-tune các mô hình Text-to-SQL trong tương lai.

VI. KẾT LUẬN (CONCLUSION)

Nghiên cứu đã chứng minh rằng việc kết hợp kiến trúc multi-agent với cơ chế Retrieval-Augmented Generation (Trustay-AI) trên cùng backbone Google Gemini Flash 2.0 mang lại lợi thế rõ rệt cho bài toán Text-to-SQL trong môi trường nghiệp vụ. Trustay-AI đạt **80%** độ chính xác thực thi, vượt xa baseline prompt thuần túy (45.8%).

Quan trọng hơn, chiến thắng này không chỉ là về độ chính xác. Baseline thất bại nặng nề do **không thể tạo ra SQL hợp lệ** (chiếm 40% số case) và **không hiểu ngữ cảnh** (0 điểm ở các case INSIGHT). Trustay-AI đã giải quyết triệt để các vấn đề này thông qua các agent chuyên biệt: Orchestrator Agent thành công trong việc nhận diện ngữ cảnh trang; Validator Agent cho thấy khả năng tự sửa lỗi cú pháp SQL theo vòng lặp; và SQL Agent áp dụng đúng các quy tắc bảo mật cấp độ cơ bản (điều mà baseline đã vi phạm).

Mặc dù vậy, hệ thống vẫn thất bại ở các bài toán logic phức tạp nhất (như logic 'VÀ' và bảo mật subquery), cho thấy đây vẫn là một thách thức lớn.

Trong tương lai, chúng tôi tập trung vào hai hướng chính: (i) tăng cường sức mạnh cho Validator Agent, chuyển nó từ một công cụ kiểm tra cú pháp thành một “kiểm định viên” logic nghiệp vụ và bảo mật, có khả năng phát hiện các lỗi trong subquery; và (ii) cải thiện RAG để xử lý logic “VÀ” cho các truy vấn liệt kê, thay vì chỉ map từ khóa đơn thuần.

TÀI LIỆU THAM KHẢO

- [1] S. Madden, “Few-shot text-to-sql translation using structure and content prompt learning,” 2023, [Online]. Available: <https://nantang.github.io/research/pubs/scprompt.pdf>
- [2] Z. Hong *et al.*, “A survey of llm-based text-to-sql,” *arXiv preprint arXiv:2406.08426*, 2024, [Online]. Available: <https://arxiv.org/abs/2406.08426>
- [3] G. Zhu, W. Shao, X. Zhu, L. Yu, J. Guo, and X. Cheng, “Text2sql: Pure fine-tuning and pure knowledge distillation,” in *Proc. North American Chapter of the Association for Computational Linguistics: Industry Track (NAACL-Industry)*, 2025, [Online]. Available: <https://aclanthology.org/2025.naacl-industry.5.pdf>
- [4] D. Klisura and A. Rios, “Unmasking database vulnerabilities: Zero-knowledge schema inference attacks in text-to-sql systems,” in *Proc. Findings of the North American Chapter of the Association for Computational Linguistics (NAACL Findings)*, 2025, [Online]. Available: <https://aclanthology.org/2025.findings-naacl.386.pdf>
- [5] C. Renggli *et al.*, “Fundamental challenges in evaluating text2sql solutions and their real-world applicability,” *arXiv preprint arXiv:2501.18197*, 2024, [Online]. Available: <https://arxiv.org/pdf/2501.18197.pdf>
- [6] A. Biswal *et al.*, “Text2sql is not enough: Unifying ai and databases with tag,” in *Proc. Conference on Innovative Data Systems Research (CIDR)*, 2025, [Online]. Available: <https://vldb.org/cidrdb/papers/2025/p11-biswal.pdf>
- [7] J. Devlin *et al.*, “Bert: Pre-training of deep bidirectional transformers for language understanding,” in *Proc. North American Chapter of the Association for Computational Linguistics: Human Language Technologies (NAACL-HLT)*, 2019, pp. 4171–4186, [Online]. Available: <https://aclanthology.org/N19-1423/>
- [8] V. Karpukhin *et al.*, “Dense passage retrieval for open-domain question answering,” in *Proc. Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 6769–6781, [Online]. Available: <https://aclanthology.org/2020.emnlp-main.550/>
- [9] P. Lewis *et al.*, “Retrieval-augmented generation for knowledge-intensive nlp tasks,” in *Proc. Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, 2020, pp. 9459–9474, [Online]. Available: <https://arxiv.org/abs/2005.11401>
- [10] P. Gurawa and A. Dharmik, “Balancing content size in rag-text2sql system,” *arXiv preprint arXiv:2502.15723*, 2025, [Online]. Available: <https://arxiv.org/pdf/2502.15723.pdf>
- [11] Y. Wang *et al.*, “Rb-sql: A retrieval-based llm framework for text-to-sql,” *arXiv preprint arXiv:2407.08273*, 2024, [Online]. Available: <https://arxiv.org/html/2407.08273v1>
- [12] J. Becker *et al.*, “Mallm: Multi-agent large language models framework,” *arXiv preprint arXiv:2509.11656*, 2025, [Online]. Available: <https://arxiv.org/abs/2509.11656>
- [13] K. T. Tran *et al.*, “Multi-agent collaboration mechanisms: A survey of llms,” *arXiv preprint arXiv:2501.06322*, 2025, [Online]. Available: <https://arxiv.org/abs/2501.06322>